# Corso di Laboratorio di Fisica Computazionale 1

Appunti ed Esempi

*Basato sulle lezioni di*
**Prof. Carlo Mancini Terracciano**

---

Autore: **Liam Ferretti**
28 ottobre 2026

# Indice

**1 Informazioni sul Corso** **5**
1.1 Comandi UNIX / Bash . . . . . 5
1.2 Git . . . . . 5
1.3 Il `main` in C . . . . . 6

**2 Metodi Numerici** **7**
2.1 Metodo di Eulero . . . . . 7
2.1.1 Interpretazione integrale . . . . . 8
2.1.2 EDO del secondo ordine . . . . . 8
2.1.3 Errore . . . . . 8
2.1.4 Verifiche della bontà dell'integrazione . . . . . 8
2.1.5 Come trovare l'andamento dell'errore in funzione di $\Delta t$ . . . . . 9
2.1.6 Esempio: oscillatore armonico . . . . . 9
2.1.7 Forma matriciale . . . . . 10
2.2 Metodo di Eulero-Cromer . . . . . 10
2.3 Metodo di Verlet . . . . . 10
2.4 Metodo Velocity Verlet . . . . . 10
2.4.1 Forma matriciale . . . . . 11
2.4.2 Stabilità . . . . . 11
2.4.3 Perché Velocity Verlet è molto meglio di Eulero-Cromer . . . . . 11
2.4.4 Forma con il mezzo passo . . . . . 12
2.5 Metodi di Runge-Kutta . . . . . 12
2.5.1 Runge-Kutta del secondo ordine (RK2) . . . . . 12
2.5.2 Runge-Kutta del quarto ordine (RK4) . . . . . 13
2.5.3 Costo computazionale e scelta dell'integratore . . . . . 14

# Capitolo 1

# Informazioni sul Corso

- Contatto: `carlo.mancini-terracciano@uniroma1.it`.
- **Modalità d'esame**:
  - fino a 3 punti bonus in funzione della presenza in laboratorio e alle esercitazioni;
  - 2 prove individuali al computer: 1) integrazione numerica di EDO; 2) processi stocastici.
- Libro di testo: *Programmazione Scientifica* (da leggere con atteggiamento critico!).

## 1.1 Comandi UNIX / Bash

- `ls`: elenca il contenuto della directory;
- `cd`: cambia directory (`cd ..` torna al livello superiore);
- `mkdir`: crea una directory;
- `pwd`: restituisce il path assoluto della directory corrente;
- `man <cmd>`: restituisce il manuale del comando;
- `less file`: visualizza un file un pezzo alla volta;
- `fg`: riporta in primo piano un processo sospeso o in background.

## 1.2 Git

- `git config --global core.editor "emacs -nw"`: imposta l'editor;
- `git init`: inizializza un repository (crea la cartella `.git`);
- `git add .` aggiunge tutta la directory, `git add nomefile` solo quel file;
- `git commit -m "msg"`: registra le modifiche con un messaggio;
- `.gitignore`: file con l'elenco dei file da non tracciare.

## 1.3 Il `main` in C

```
int main(int argc, char *argv[]) {
    ...
}
```

- Si usa `void` come argomento se non si vogliono fornire input da riga di comando.
- Si usa `int argc, char *argv[]` per leggere gli argomenti: `argv` è un array di stringhe (array di array di `char`) e `argc` è il numero di stringhe, cioè la lunghezza di `argv` (il primo elemento è il nome del programma).

Un'alternativa a `fprintf` per scrivere su file è l'**operatore di redirezione** della shell: `./codice > file_output`.

# Capitolo 2

# Metodi Numerici

I metodi numerici permettono di approssimare edo che non siamo in grado di risolvere analiticamente, come ad esempio quella del pendolo:

$$\ddot{\theta} + \frac{g}{L}\sin\theta = 0 \tag{2.1}$$

che sappiamo risolvere solo nell'approssimazione $\sin\theta \simeq \theta$. Tale edo non ha una soluzione elementare, e quindi è necessario approssimarla tramite metodi numerici; in generale si scrive $\ddot{\theta} = f(\theta, t)$.

Per l'uso di metodi numerici è necessario passare dal continuo al discreto, quindi discretizzare il tempo, introducendo il passo temporale:

$$\Delta t \implies t_n = \Delta t \cdot n + t_0 \tag{2.2}$$

La derivata, definita come limite del rapporto incrementale per $h \to 0$, viene approssimata con un incremento finito:

$$f'(x) = \lim_{h\to 0} \frac{f(x+h) - f(x)}{h} \quad \longrightarrow \quad \dot{y}(t) \approx \frac{y(t+\Delta t) - y(t)}{\Delta t} \tag{2.3}$$

## 2.1 Metodo di Eulero

Il metodo di approssimazione numerica più semplice è quello di Eulero, considerando un problema di Cauchy di un EDO al primo ordine, del tipo:

$$\dot{y} = f(y(t), t), \quad y(t=0) = y_0 \tag{2.4}$$

Definendo la discretizzazione del tempo tramite un passo $\Delta t$, allora vale che:

$$\dot{y}(t_n) \approx \frac{y(t_n + \Delta t) - y(t_n)}{\Delta t} = \frac{y(t_{n+1}) - y(t_n)}{\Delta t} \tag{2.5}$$

$$\frac{y(t_{n+1}) - y(t_n)}{\Delta t} \approx f(y(t_n), t_n) \tag{2.6}$$

$$y(t_{n+1}) \approx y(t_n) + f(y(t_n), t_n)\,\Delta t \tag{2.7}$$

cioè, ad ogni passo, si usa la derivata calcolata all'*estremo sinistro* dell'intervallo:

$$\boxed{x_{n+1} = x_n + \dot{x}_n\,\Delta t} \tag{2.8}$$

### 2.1.1 Interpretazione integrale

Integrando esattamente l'edo $\dot{x} = f(x,t)$ tra $t_n$ e $t_{n+1}$:

$$x_{n+1} - x_n = \int_{t_n}^{t_{n+1}} f(x,t)\,dt \tag{2.9}$$

$$x_{n+1} = x_n + \frac{\Delta t}{\Delta t}\int_{t_n}^{t_{n+1}} f(x,t)\,dt = x_n + \Delta t\,\langle f(x,t)\rangle \tag{2.10}$$

dove $\langle f\rangle$ è il **valor medio** di $f$ sull'intervallo. Questa relazione è *esatta*: se sapessimo calcolare il valor medio di $f$ sull'intervallo avremmo risolto l'edo. Tutti i metodi numerici sono quindi diversi modi di *stimare* $\langle f\rangle$. Eulero lo approssima con il valore all'estremo sinistro:

$$\langle f\rangle \simeq f(x_n,t_n) \implies x_{n+1} = x_n + \dot{x}_n\,\Delta t \tag{2.11}$$

### 2.1.2 EDO del secondo ordine

Se il problema di Cauchy è del secondo ordine, $\ddot{x} = \varphi(x,\dot{x},t)$, lo si riscrive come sistema di due edo del primo ordine:

$$\ddot{x} = \varphi \implies \begin{cases}\dot{x} = v\\ \dot{v} = \varphi\end{cases} \tag{2.12}$$

Sviluppando in serie di Taylor:

$$v(t+\Delta t) = v(t) + \dot{v}(t)\Delta t + O(\Delta t^2) = v(t) + \varphi(t)\Delta t + O(\Delta t^2) \tag{2.13}$$

$$x(t+\Delta t) = x(t) + v(t)\Delta t + O(\Delta t^2) \tag{2.14}$$

da cui il **metodo di Eulero**:

$$\begin{cases}x_{n+1} = x_n + v_n\,\Delta t\\ v_{n+1} = v_n + \varphi_n\,\Delta t\end{cases} \tag{2.15}$$

### 2.1.3 Errore

- Errore locale (per passo): $\varepsilon \propto \Delta t^2$.
- Errore globale (totale): il numero di passi è $\propto \frac{1}{\Delta t}$, quindi $\varepsilon \propto \Delta t^2 \cdot \frac{1}{\Delta t} = \Delta t$.

Eulero è quindi un metodo del *primo ordine*.

### 2.1.4 Verifiche della bontà dell'integrazione

- **Conservazione dell'energia**: se il sistema è conservativo si può stimare l'errore relativo sull'energia:
$$\varepsilon = \frac{|E_i - E_f|}{E_i} \tag{2.16}$$
- **Stabilità in funzione di $\Delta t$**: si riduce $\Delta t$ e si calcola la differenza tra le soluzioni ottenute con passi successivi, $\varepsilon = |f_i - f_{i+1}|$. Al diminuire di $\Delta t$ l'errore diminuisce, fino a un passo $\Delta t_c$ sotto il quale l'errore smette di decrescere e oscilla, dominato dagli errori di arrotondamento.
- **Spazio delle fasi**: se si sa a priori che il moto è periodico, la traiettoria nello spazio delle fasi deve essere una curva chiusa.

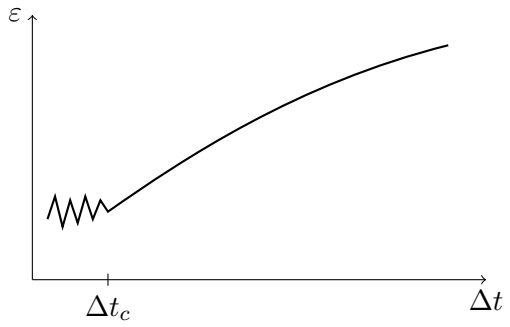



Figura 2.1: Andamento qualitativo dell'errore al variare del passo d'integrazione.

### 2.1.5 Come trovare l'andamento dell'errore in funzione di $\Delta t$

Se $\varepsilon \propto \Delta t^n$, cioè $\varepsilon = K\Delta t^n$, passando ai logaritmi:

$$\ln \varepsilon = \ln K + n \ln \Delta t \tag{2.17}$$

che è una retta $y = q + mx$ con $y = \ln \varepsilon$, $x = \ln \Delta t$, pendenza $m = n$ e intercetta $q = \ln K$. Un fit lineare in scala log-log fornisce quindi l'ordine $n$ del metodo.

### 2.1.6 Esempio: oscillatore armonico

$$\ddot{x} + \omega^2 x = 0 \implies \begin{cases} \dot{x} = v \\ \dot{v} = -\omega^2 x \end{cases} \tag{2.18}$$

con energia (per $m = 1$):

$$E = \frac{1}{2}mv^2 + \frac{1}{2}m\omega^2 x^2 \tag{2.19}$$

La soluzione analitica è:

$$x(t) = A_0 \cos(\omega t + \varphi_0), \qquad v(t) = -\omega A_0 \sin(\omega t + \varphi_0) \tag{2.20}$$

Imponendo le condizioni iniziali:

$$x_0 = A_0 \cos \varphi_0, \qquad v_0 = -\omega A_0 \sin \varphi_0 \tag{2.21}$$

$$\frac{x_0}{\cos \varphi_0} = \frac{v_0}{-\omega \sin \varphi_0} \implies \tan \varphi_0 = -\frac{v_0}{\omega x_0} \implies \varphi_0 = \arctan\left(-\frac{v_0}{\omega x_0}\right) \tag{2.22}$$

$$A_0 = \frac{x_0}{\cos \varphi_0} = \sqrt{x_0^2 + \frac{v_0^2}{\omega^2}} \tag{2.23}$$

*Osservazione* 2.1.1. Negli appunti a mano compare $\varphi_0 = \arctan\left(\frac{v_0}{x_0}\right)$: il risultato corretto ha il segno meno e il fattore $\omega$ (per $\omega = 1$ resta $\varphi_0 = -\arctan\frac{v_0}{x_0}$). Inoltre l'arctan restituisce valori in $\left(-\frac{\pi}{2}, \frac{\pi}{2}\right)$: nel codice conviene usare `atan2(-v0/omega, x0)` per ottenere il quadrante corretto.

**Esempio 2.1.2** (Primo passo di Eulero)**.** Con $x_0 = 2$, $v_0 = 1$, $\omega^2 = 1$, $\Delta t = 0.1$:

$$v_1 = v_0 + \varphi_0(x_0)\Delta t = 1 - 0.2 = 0.8 \tag{2.24}$$

$$x_1 = x_0 + v_0 \Delta t = 2 + 0.1 = 2.1 \tag{2.25}$$

**Non conservazione dell'energia.** Per l'oscillatore armonico con Eulero si trova:

$$E_{n+1} = E_n\left(1 + \omega^2 \Delta t^2\right) \implies E_n = E_0\left(1 + \omega^2 \Delta t^2\right)^n = E_0\left(1 + \omega^2 \Delta t^2\right)^{\frac{t_n}{\Delta t}} \tag{2.26}$$

cioè l'energia cresce esponenzialmente: il metodo di Eulero è instabile per l'oscillatore armonico.

### 2.1.7 Forma matriciale

Per l'oscillatore armonico ($a_n = -\omega^2 x_n$) Eulero si scrive come:

$$\begin{pmatrix} x_{n+1} \\ v_{n+1} \end{pmatrix} = \begin{pmatrix} 1 & \Delta t \\ -\omega^2 \Delta t & 1 \end{pmatrix} \begin{pmatrix} x_n \\ v_n \end{pmatrix} \tag{2.27}$$

Il determinante della matrice è $1 + \omega^2 \Delta t^2 > 1$: l'area nello spazio delle fasi cresce ad ogni passo (coerentemente con la crescita dell'energia).

## 2.2 Metodo di Eulero-Cromer

Una piccola modifica di Eulero consiste nell'aggiornare prima la velocità e usare la velocità *aggiornata* per la posizione:

$$\begin{cases} v_{n+1} = v_n + \varphi_n \, \Delta t \\ x_{n+1} = x_n + v_{n+1} \, \Delta t \end{cases} \tag{2.28}$$

Per l'oscillatore armonico la matrice di evoluzione ha determinante 1, e l'energia oscilla invece di divergere.

## 2.3 Metodo di Verlet

Sviluppando in serie di Taylor in avanti e all'indietro:

$$x_{n+1} = x_n + \dot{x}_n \Delta t + \frac{1}{2} \ddot{x}_n \Delta t^2 + O(\Delta t^3) \tag{2.29}$$

$$x_{n-1} = x_n - \dot{x}_n \Delta t + \frac{1}{2} \ddot{x}_n \Delta t^2 + O(\Delta t^3) \tag{2.30}$$

Sommando, i termini dispari si cancellano:

$$x_{n+1} + x_{n-1} = 2x_n + \ddot{x}_n \Delta t^2 + O(\Delta t^4) \implies \boxed{x_{n+1} = 2x_n - x_{n-1} + a_n \Delta t^2} \tag{2.31}$$

Sottraendo invece:

$$x_{n+1} - x_{n-1} = 2\dot{x}_n \Delta t + O(\Delta t^3) \implies v_n = \frac{x_{n+1} - x_{n-1}}{2\Delta t} + O(\Delta t^2) \tag{2.32}$$

Se la forza non dipende dalla velocità, la posizione ha un errore piccolo (i termini del terzo ordine si cancellano, l'errore locale è $O(\Delta t^4)$), mentre la velocità ha un errore del secondo ordine. L'energia in ogni caso ha errore $\propto \Delta t^2$.

## 2.4 Metodo Velocity Verlet

Partendo dallo sviluppo di Taylor della posizione:

$$x(t + \Delta t) = x(t) + v(t)\Delta t + \frac{1}{2} a(t) \Delta t^2 + O(\Delta t^3) \implies x_{n+1} = x_n + v_n \Delta t + \frac{1}{2} \varphi_n \Delta t^2 \tag{2.33}$$

Per la velocità vale $\dot{v} = a$:

$$\int_t^{t+\Delta t} \frac{dv}{dt'} \, dt' = \int_t^{t+\Delta t} a(t') \, dt' \tag{2.34}$$

e stimando l'integrale a destra con il **metodo dei trapezi**:

$$v_{n+1} - v_n = \frac{a_{n+1} + a_n}{2} \Delta t \implies v_{n+1} = v_n + \frac{1}{2} \left( a_{n+1} + a_n \right) \Delta t \tag{2.35}$$

Il metodo **Velocity Verlet** è quindi:

$$\boxed{\begin{cases} x_{n+1} = x_n + v_n \Delta t + \frac{1}{2} a_n \Delta t^2 \\ v_{n+1} = v_n + \frac{1}{2} \left( a_{n+1} + a_n \right) \Delta t \end{cases}} \tag{2.36}$$

dove $a_{n+1} = \varphi(x_{n+1})$: la forza **non deve dipendere dalla velocità**, perché per calcolare $a_{n+1}$ servirebbe $v_{n+1}$, che non è ancora noto.

### 2.4.1 Forma matriciale

Per l'oscillatore armonico $\varphi = -\omega^2 x$:

$$x_{n+1} = x_n + v_n \Delta t - \frac{1}{2} \omega^2 x_n \Delta t^2 \tag{2.37}$$

$$v_{n+1} = v_n - \frac{1}{2} \omega^2 \Delta t \left( x_n + v_n \Delta t - \frac{1}{2} \omega^2 x_n \Delta t^2 + x_n \right) \tag{2.38}$$

$$= v_n \left( 1 - \frac{1}{2} \omega^2 \Delta t^2 \right) + x_n \left( \frac{1}{4} \omega^4 \Delta t^3 - \omega^2 \Delta t \right) \tag{2.39}$$

$$\begin{pmatrix} x_{n+1} \\ v_{n+1} \end{pmatrix} = \begin{pmatrix} 1 - \frac{1}{2} \omega^2 \Delta t^2 & \Delta t \\ \frac{1}{4} \omega^4 \Delta t^3 - \omega^2 \Delta t & 1 - \frac{1}{2} \omega^2 \Delta t^2 \end{pmatrix} \begin{pmatrix} x_n \\ v_n \end{pmatrix} \tag{2.40}$$

Il determinante è:

$$\det M = \left( 1 - \frac{1}{2} \omega^2 \Delta t^2 \right)^2 - \frac{1}{4} \omega^4 \Delta t^4 + \omega^2 \Delta t^2 \tag{2.41}$$

$$= 1 + \frac{1}{4} \omega^4 \Delta t^4 - \omega^2 \Delta t^2 - \frac{1}{4} \omega^4 \Delta t^4 + \omega^2 \Delta t^2 = 1 \tag{2.42}$$

Quindi l'area nello spazio $(x, v)$ si conserva: l'algoritmo è **simplettico**.

### 2.4.2 Stabilità

Scrivendo $W_n = (x_n, v_n)^T$ come combinazione degli autovettori $U_1, U_2$ di $M$:

$$W_{n+1} = M W_n = M \left( c_1 U_1 + c_2 U_2 \right) = \lambda_1 c_1 U_1 + \lambda_2 c_2 U_2 \tag{2.43}$$

Se $\lambda_1$ o $\lambda_2$ hanno modulo $> 1$ la soluzione esplode. Il polinomio caratteristico è:

$$\lambda^2 - \left( 2 - \omega^2 \Delta t^2 \right) \lambda + 1 = 0 \tag{2.44}$$

Poiché $\lambda_1 \lambda_2 = 1$, gli autovalori hanno modulo unitario (complessi coniugati) se il discriminante è negativo, cioè $|2 - \omega^2 \Delta t^2| < 2$. Quindi:

$$\Delta t < \frac{2}{\omega} \implies \text{l'algoritmo è stabile} \tag{2.45}$$

### 2.4.3 Perché Velocity Verlet è molto meglio di Eulero-Cromer

Sviluppando l'accelerazione:

$$a_{n+1} = a_n + \dot{a}_n \Delta t + \frac{\ddot{a}_n}{2} \Delta t^2 + O(\Delta t^3) \tag{2.46}$$

e sostituendo nella velocità di Velocity Verlet:

$$v_{n+1} = v_n + a_n \Delta t + \frac{1}{2} \dot{a}_n \Delta t^2 + \frac{1}{4} \ddot{a}_n \Delta t^3 \tag{2.47}$$

mentre lo sviluppo esatto è:

$$v(t_{n+1}) = v(t_n) + a(t_n)\Delta t + \frac{1}{2}\dot{a}(t_n)\Delta t^2 + \frac{1}{6}\ddot{a}(t_n)\Delta t^3 \tag{2.48}$$

quindi:

$$v_{n+1} - v(t_{n+1}) = \frac{1}{4}\ddot{a}_n\Delta t^3 - \frac{1}{6}\ddot{a}_n\Delta t^3 + O(\Delta t^4) = \frac{1}{12}\ddot{a}_n\Delta t^3 + O(\Delta t^4) \tag{2.49}$$

L'errore locale è $O(\Delta t^3)$, quindi l'errore globale è $O(\Delta t^2)$: Velocity Verlet è un metodo del *secondo ordine*.

*Osservazione* 2.4.1. Velocity Verlet (VV) è pensato per equazioni differenziali del **secondo ordine**: posizione e velocità vengono aggiornate in modo diverso. Eulero ed Eulero-Cromer sono invece metodi per equazioni del **primo ordine**, applicati due volte al sistema equivalente (posizione e velocità trattate allo stesso modo).

### 2.4.4 Forma con il mezzo passo

Raccogliendo i termini:

$$x_{n+1} = x_n + \left(v_n + \frac{1}{2}a_n\Delta t\right)\Delta t \tag{2.50}$$

$$v_{n+1} = \left(v_n + \frac{1}{2}a_n\Delta t\right) + \frac{1}{2}a_{n+1}\Delta t \tag{2.51}$$

e definendo la velocità al mezzo passo $v_{n+\frac{1}{2}} \equiv v_n + a_n\frac{\Delta t}{2}$:

$$\begin{cases} v_{n+\frac{1}{2}} = v_n + a_n\frac{\Delta t}{2} \\ x_{n+1} = x_n + v_{n+\frac{1}{2}}\Delta t \\ v_{n+1} = v_{n+\frac{1}{2}} + \frac{1}{2}a_{n+1}\Delta t \end{cases} \tag{2.52}$$

In questa forma basta calcolare la forza una sola volta per passo ($a_{n+1}$ viene riutilizzata come $a_n$ al passo successivo).

## 2.5 Metodi di Runge-Kutta

Come si procede se l'accelerazione dipende dalla velocità, $a = a(v)$? Ad esempio per l'oscillatore smorzato:

$$\ddot{x} = -\omega^2 x - \frac{\gamma}{m}\dot{x} \tag{2.53}$$

Velocity Verlet non è applicabile, perché per aggiornare $v$ servirebbe l'accelerazione alla fine del passo, che dipende dalla velocità ancora incognita. Si possono invece usare i metodi di **Runge-Kutta**, una classe di algoritmi numerici per edo del primo ordine che usano *solo* l'informazione disponibile all'inizio del passo (come Eulero), ma la combinano in modo da stimare meglio il valor medio della derivata sull'intervallo.

### 2.5.1 Runge-Kutta del secondo ordine (RK2)

L'idea è, invece di fare tutto il passo seguendo la derivata nel punto iniziale (Eulero):

1. si usa la derivata nel punto iniziale per muoversi di *mezzo passo*;
2. si calcola la derivata in questo punto intermedio (stimato);
3. si torna al punto iniziale e si usa la derivata a metà per fare l'intero passo.

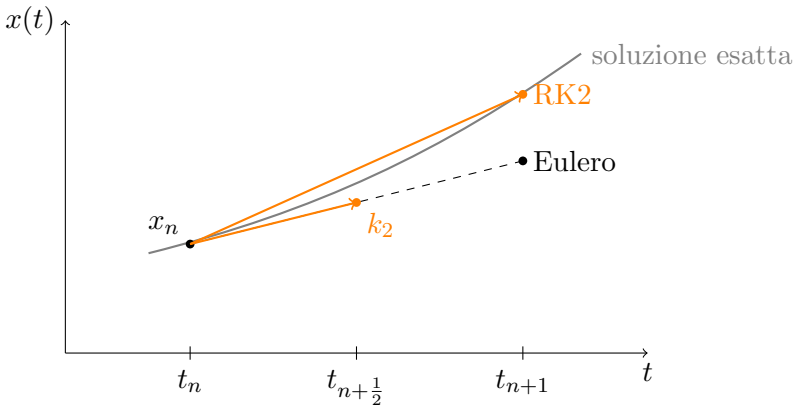



Figura 2.2: Schema di RK2: la derivata stimata a metà passo viene usata per l'intero passo.

Data l'edo $\dot{x} = \varphi(x, t)$:

$$k_1 = \varphi(x_n, t_n) \tag{2.54}$$

$$k_2 = \varphi\left(x_n + k_1\frac{\Delta t}{2}, t_n + \frac{\Delta t}{2}\right) = \varphi\left(x_n + \dot{x}_n\frac{\Delta t}{2}, t_n + \frac{\Delta t}{2}\right) \tag{2.55}$$

$$\boxed{x_{n+1} = x_n + k_2\Delta t} \tag{2.56}$$

- $k_1$ è la derivata all'inizio dell'intervallo;
- $x_n + k_1\frac{\Delta t}{2}$ è la stima (fatta con Eulero) della posizione a metà passo;
- $k_2$ è quindi la stima della derivata a metà intervallo, cioè la stima di $\langle\varphi\rangle$ (non si fa la media tra derivate: si prende la derivata a metà).

Ricordando che $x_{n+1} = x_n + \Delta t\,\langle\varphi\rangle$ è esatta, Eulero approssima $\langle\varphi\rangle$ con il valore all'inizio dell'intervallo, mentre RK2 lo approssima con il valore (stimato) a metà intervallo, che è statisticamente più vicino al valor medio vero.

*Osservazione* 2.5.1. Il punto intermedio non è "calcolato" ma *stimato* usando solo informazioni all'istante $t_n$. Per questo Runge-Kutta, a differenza di Velocity Verlet, funziona anche se la forza dipende dalla velocità.

**EDO del secondo ordine.** Si riscrive l'edo come sistema di due edo del primo ordine:

$$\begin{cases}\dot{x} = v \\ \dot{v} = \xi(x, v, t)\end{cases} \tag{2.57}$$

e si applica RK2 a entrambe ("si usa due volte RK2"), una per la posizione e una per la velocità, calcolando i coefficienti $k$ di entrambe le equazioni in parallelo: quando lo si applica all'equazione per $v$, il ruolo di $k_1$ è svolto dall'accelerazione all'inizio del passo.

### 2.5.2 Runge-Kutta del quarto ordine (RK4)

$$k_1 = \varphi(x_n, t_n) \tag{2.58}$$

$$k_2 = \varphi\left(x_n + k_1\frac{\Delta t}{2}, t_n + \frac{\Delta t}{2}\right) \tag{2.59}$$

$$k_3 = \varphi\left(x_n + k_2\frac{\Delta t}{2}, t_n + \frac{\Delta t}{2}\right) \tag{2.60}$$

$$k_4 = \varphi\left(x_n + k_3\Delta t, t_n + \Delta t\right) \tag{2.61}$$

$$\boxed{x_{n+1} = x_n + \left(\frac{1}{6}k_1 + \frac{1}{3}k_2 + \frac{1}{3}k_3 + \frac{1}{6}k_4\right)\Delta t} \tag{2.62}$$

dove la combinazione lineare tra parentesi è la stima di $\langle\varphi\rangle$. Tutti i $k$ sono stime della derivata nell'intervallo:

- $k_1$: derivata all'inizio dell'intervallo;
- $k_2$: stima della derivata a metà intervallo, arrivando a metà con Eulero (cioè con la derivata iniziale $k_1$);
- $k_3$: di nuovo una stima della derivata a metà intervallo, ma arrivando a metà partendo da $x_n$ con la derivata $k_2$;
- $k_4$: stima della derivata alla *fine* del passo, arrivando a $t_n + \Delta t$ con la derivata $k_3$ (per questo compare $\Delta t$ e non $\frac{\Delta t}{2}$).

*Osservazione* 2.5.2. Negli appunti a mano il titolo riporta "RK3" e in $k_4$ compare $k_3\frac{\Delta t}{2}$: lo schema scritto è RK4, e in $k_4$ il passo è intero, $x_n + k_3\Delta t$, coerentemente con $t_n + \Delta t$.

I coefficienti della combinazione lineare derivano da (lunghi) conti con lo sviluppo di Taylor: si scrive la combinazione lineare con coefficienti incogniti e si impone che coincida con lo sviluppo in serie di Taylor di $x(t_n + \Delta t)$ fino al quarto ordine incluso. La simmetria dei coefficienti viene fuori dallo sviluppo di Taylor. In linea di principio si possono costruire Runge-Kutta di ordine superiore, ma raramente si va oltre il quarto ordine.

### 2.5.3 Costo computazionale e scelta dell'integratore

Il costo di un passo si misura in numero di valutazioni della funzione $\varphi$ (la forza):

- RK4 richiede 4 valutazioni per la posizione e 4 per la velocità: è più preciso ma più costoso;
- Velocity Verlet, implementato nella forma con mezzo passo, richiede una sola nuova valutazione della forza per passo.

Una domanda utile per le esercitazioni: quanto deve essere più piccolo il passo d'integrazione di Eulero rispetto a RK2 (o RK4) per avere lo stesso errore globale, e quale metodo risulta quindi computazionalmente più conveniente?

Ci sono sostanzialmente due famiglie di uso:

- se si simula un sistema in cui l'energia si conserva e la forza non dipende dalla velocità (ad esempio in dinamica molecolare), si usa tipicamente **Velocity Verlet**, che è simplettico;
- se la forza dipende dalla velocità (attrito, particelle cariche in campo magnetico) o non lo si sa a priori, si usa **Runge-Kutta** (tipicamente RK4).